

Supplemental Material: *Diagonal Hessian Approximation Based on Conjugacy Condition for Noisy Derivative-Free Optimization Problems in High Dimensions*

Morteza Kimiaei

*Fakultät für Mathematik, Universität Wien
Oskar–Morgenstern–Platz 1, A–1090 Wien, Austria
email: kimiaeim83@univie.ac.at
<http://www.mat.univie.ac.at/~kimiaei>*

Saman Babaie–Kafaki

*Faculty of Engineering, Free University of Bozen–Bolzano
NOI Techpark, Via Bruno Buozzi 1, 39100 Bolzano (BZ), Italy
email: saman.babaiekafaki@unibz.it*

June 18, 2026

Contents

1	Implementation Issue	2
2	Computational Cost of the Diagonal Mutation Scaling	5
3	A Comparison Among DMADFO Variants	7
4	Tables for Comparison of the Most Robust DMADFO Variant with MADFO	9
5	Tables for Comparison of the Most Robust DMADFO Variant with LMMAES	10

This supplementary material provides additional implementation details, complexity comparisons, and numerical evidence supporting the proposed diagonal variant of **MADFO**, called **DMADFO**.

Section 1 explains how the proposed diagonal conjugacy-based scaling is incorporated into the mutation phase of **MADFO** and discusses the main tuning parameters, including the conjugacy-penalty parameter, reciprocal regularization parameter, safeguarding bounds, population size, number of selected mutation points, and recombination weights.

Section 2 compares the storage and arithmetic costs of the proposed diagonal mutation scaling with **MADFO** [3] and **LMMAES** [4]. For context, we also mention classical **MAES** [1] and **fMAES** [2].

Section 3 compares twelve **DMADFO** variants corresponding to the penalty-parameter pairs

$$(p, z) \in \{(1, 0), (1, 1), (2, 0), (3, 0), \dots, (10, 0), (15, 0), (20, 0)\}.$$

Here, $(p, z) = (1, 0)$ gives the quadratic penalty model, $(p, z) = (1, 1)$ gives the cubic penalty model, and the remaining pairs correspond to the general higher-order model with $z = 0$. Based on these comparisons, **DMADFO12**, corresponding to $(p, z) = (20, 0)$, is selected as the representative implementation for the subsequent comparisons.

Section 4 compares **DMADFO12** with the original **MADFO** solver, which uses a full affine scaling matrix, to assess the loss caused by replacing full matrix adaptation with diagonal scaling.

Finally, Section 5 compares **DMADFO12** with **LMMAES**, which uses a limited-memory affine scaling mechanism, on a larger noisy test set. These additional comparisons support the use of **DMADFO12** as a computationally low-cost and large-scale alternative to full and limited-memory matrix adaptation.

1 Implementation Issue

The proposed diagonal scaling strategy is intended to be incorporated into the **MADFO** solver. The original **MADFO** algorithm was introduced as an effective matrix adaptation evolution strategy for noisy DFO (Derivative-Free Optimization) problems. It consists of the usual mutation, selection, and recombination phases. Still, it modifies the mutation phase by computing good mutation step sizes that are neither too small nor too large, and improves the recombination phase through a randomized nonmonotone line search and additional heuristic mechanisms for updating the best point. Numerical results reported for noisy **prince** test problems of dimensions $n \leq 20$ show that **MADFO** is particularly robust under different types and levels of noise [3].

Tuning the parameters. The proposed diagonal scaling mechanism involves only a small number of tuning parameters. The parameter $\mu > 0$, implemented as **muConj**, controls the

strength of the conjugacy penalty in

$$\mathcal{P}_{k+1}^{(2)} := \mathcal{B}_k - \frac{\mu \boldsymbol{\tau}_k^\top \mathbf{b}_k}{\mu \|\boldsymbol{\tau}_k\|^2 + \rho + 1} \mathcal{T}_k \approx \nabla^2 f(\mathbf{x}_{k+1}),$$

whose elements are defined in the underlying manuscript; that is,

$$\mathbf{p}_{k+1} := \begin{pmatrix} p_{1_{k+1}} \\ p_{2_{k+1}} \\ \vdots \\ p_{n_{k+1}} \end{pmatrix}, \quad \mathbf{p}_k := \begin{pmatrix} p_{1_k} \\ p_{2_k} \\ \vdots \\ p_{n_k} \end{pmatrix}, \quad \text{and} \quad \boldsymbol{\tau}_k := \begin{pmatrix} \tau_1^{(k-1,k)} \\ \tau_2^{(k-1,k)} \\ \vdots \\ \tau_n^{(k-1,k)} \end{pmatrix} = \mathbf{d}_{k-1} \odot \mathbf{d}_k,$$

and also,

$$\begin{aligned} \mathbf{b}_k &:= \frac{1}{1 + \rho} \left(p_{1_k} + \frac{\rho}{p_{1_k}}, p_{2_k} + \frac{\rho}{p_{2_k}}, \dots, p_{n_k} + \frac{\rho}{p_{n_k}} \right)^\top, \\ \mathcal{B}_k &:= \frac{1}{1 + \rho} \text{diag} \left(p_{1_k} + \frac{\rho}{p_{1_k}}, p_{2_k} + \frac{\rho}{p_{2_k}}, \dots, p_{n_k} + \frac{\rho}{p_{n_k}} \right), \\ \mathcal{T}_k &:= \text{diag} \left(\tau_1^{(k-1,k)}, \tau_2^{(k-1,k)}, \dots, \tau_n^{(k-1,k)} \right). \end{aligned}$$

Larger values of μ force the diagonal approximation to satisfy the conjugacy condition more strongly, while smaller values produce smoother and more conservative scaling updates.

In noisy problems, excessively large values of μ may overfit unreliable generated directions; hence moderate values are preferable. In practice, one may choose

$$\mu \in [\mu_{\min}, \mu_{\max}], \quad 0 < \mu_{\min} \leq \mu_{\max} < \infty.$$

For example, fixed values such as $\mu \in [10^{-2}, 10^2]$ may be used in preliminary implementations, while adaptive choices can be considered according to the observed violation of the conjugacy condition. In the present implementation, the default value is $\mu = 1$.

The parameter $\rho \geq 0$, implemented as `rhoConj`, controls the influence of the reciprocal regularization term through the vector $\mathbf{b}_k$. A small value of ρ gives more weight to the least-change update from $\mathbf{p}_k$, whereas a larger value pushes the diagonal entries toward reciprocal consistency and improves numerical stability. A reasonable admissible range is of the form

$$\rho \in [\rho_{\min}, \rho_{\max}], \quad 0 \leq \rho_{\min} \leq \rho_{\max} < \infty.$$

The case $\rho = 0$ is also allowed when reciprocal regularization is not required. In numerical experiments, one may start with $\rho \in [10^{-4}, 1]$, since very large values of ρ may force the scaling too strongly toward the reciprocal term. In the present implementation, the default value is $\rho = 10^{-2}$.

The safeguarding constants $p_{\min}$ and $p_{\max}$, implemented as `pmin` and `pmax`, are used to keep the diagonal entries of the Hessian approximation bounded away from zero, in the sense of

$$0 < p_{\min} \leq p_{i_{k+1}} \leq p_{\max} < \infty, \quad i = 1, 2, \dots, n.$$

These bounds are essential because the mutation scaling matrix D_{k+1} contains the inverse square roots of the diagonal entries of $\mathcal{P}_{k+1}$. Therefore, $p_{\min}$ prevents excessively large mutation variances, while $p_{\max}$ prevents excessively small mutation variances. In practice, the interval $[p_{\min}, p_{\max}]$ should be chosen wide enough to allow adaptation but narrow enough to avoid numerical instability; for example,

$$(p_{\min}, p_{\max}) = (10^{-8}, 10^8),$$

or less aggressive bounds such as

$$(p_{\min}, p_{\max}) = (10^{-4}, 10^4)$$

may be used when stronger stabilization is desired.

The population size λ , the number of selected candidates μ_{sel} , and the recombination weights w_r may be chosen according to the standard settings used in MAES-type methods. Typically,

$$\lambda \geq 2, \quad 1 \leq \mu_{\text{sel}} \leq \lambda, \quad w_r > 0, \quad \text{and} \quad \sum_{r=1}^{\mu_{\text{sel}}} w_r = 1.$$

For example, one may use the common choice

$$\lambda = 4 + \lfloor 3 \log n \rfloor, \quad \text{and} \quad \mu_{\text{sel}} = \lfloor \lambda/2 \rfloor,$$

together with positive logarithmic recombination weights.

The complete list of default tuning parameters used in the implementation is given in Table 1. The first column gives the notation used in the paper, the second column gives the corresponding field name in the code, and the third column gives the default value.

Table 1: Default tuning parameters used in the DMADFO implementation.

Parameter in the paper	Parameter in the code	Default value
Algorithmic variant	<code>ver</code>	0
Population size λ	<code>lambda</code>	$4 + \lfloor 3 \log n \rfloor$
Number of selected candidates μ_{sel}	<code>mu</code>	$\lfloor \text{lambda}/2 \rfloor$
Step-size expansion parameter γ_E	<code>gammaE</code>	4
Maximum extrapolation count E	<code>E</code>	∞
Initial step-size σ_0	<code>sigmainit</code>	1
Line-search parameter γ	<code>gamma</code>	10^{-12}
Minimum step-size $\sigma_{\min}$	<code>sigmamin</code>	10^{-2}
Maximum step-size $\sigma_{\max}$	<code>sigmamax</code>	0.5
Nonmonotone memory factor κ	<code>kappa</code>	5
Memory length N	<code>N</code>	10
Conjugacy penalty μ	<code>muConj</code>	1
Reciprocal regularization parameter ρ	<code>rhoConj</code>	10^{-2}
Lower diagonal safeguard $p_{\min}$	<code>pmin</code>	10^{-8}
Upper diagonal safeguard $p_{\max}$	<code>pmax</code>	10^8
Diagonal formula index	<code>diagFormula</code>	4
Penalty parameter p	<code>penaltyP</code>	20
Penalty parity parameter z	<code>penaltyZ</code>	0

The default value `diagFormula` = 4 corresponds to the general higher-order penalty model. In this case, the penalty exponent is $q = 2p + z$, where $p = \text{penaltyP}$ and $z = \text{penaltyZ} \in \{0, 1\}$. With the default choices `penaltyP` = 20 and `penaltyZ` = 0, the implemented default exponent is $q = 40$. The quadratic penalty model, corresponding to $(p, z) = (1, 0)$, can be selected by setting `diagFormula` = 2, while the cubic penalty model, corresponding to $(p, z) = (1, 1)$, can be selected by setting `diagFormula` = 3. Thus, the additional parameters introduced by the proposed diagonal scaling have direct numerical interpretations related to conjugacy enforcement, reciprocal regularization, safeguarding, and the order of the penalty model.

2 Computational Cost of the Diagonal Mutation Scaling

The computational cost of the proposed mutation mechanism is modest and should be compared with the internal costs of existing matrix-adaptation strategies. We distinguish between the cost of generating mutation directions and the cost of updating the matrix or scaling information. In a classical dense MAES [1] implementation, mutation directions are

generated by

$$\mathbf{d}_k^{(r)} = M_k \mathbf{z}_k^{(r)}, \quad r = 1, 2, \dots, \lambda,$$

where $M_k \in \mathbb{R}^{n \times n}$ is a dense affine-scaling matrix. Thus, generating one mutation direction costs $O(n^2)$, and generating all λ mutation directions costs $O(\lambda n^2)$. In addition, the traditional dense **MAES** matrix-update formula involves matrix–matrix products and has $O(n^3)$ arithmetic cost. Fast matrix-adaptation strategies reduce this update cost. In particular, **fMAES** [2] rewrites the affine-scaling update so that already computed mutation directions are reused. As a result, the dense affine-scaling update requires $O(n^2)$ operations instead of $O(n^3)$. The original **MADFO** solver is based on this fast affine-scaling framework and therefore also has $O(n^2)$ internal dense-matrix cost and $O(n^2)$ storage. However, both **fMAES** and **MADFO** still store a dense $n \times n$ affine-scaling matrix and still require $O(n^2)$ arithmetic to apply this matrix to a distribution direction. The limited-memory **LMAES** method reduces the dense-matrix burden by storing only $m \ll n$ evolution-path vectors. Its storage and internal arithmetic costs are $O(mn)$. If $m \in O(1)$, this is linear in n ; if $m \in O(\log n)$, it becomes $O(n \log n)$; for more details see [3, Section 1.2].

In contrast, the proposed diagonal conjugacy-based scaling requires only componentwise operations and a small number of inner products. Computing

$$\boldsymbol{\tau}_k = \mathbf{d}_{k-1} \odot \mathbf{d}_k$$

requires $O(n)$ operations. The inner products $\boldsymbol{\tau}_k^\top \mathbf{b}_k$ and $\|\boldsymbol{\tau}_k\|^2$ also require $O(n)$ operations. The diagonal update, the safeguarding step, and the multiplication

$$D_{k+1} \mathbf{z}_k^{(r)}$$

are all componentwise operations. Hence, the construction of the diagonal scaling matrix costs $O(n)$ per iteration, and the generation of each mutation direction costs $O(n)$. Therefore, for a population of size λ , the total mutation-generation cost is $O(\lambda n)$, in addition to the $O(n)$ cost of updating the diagonal approximation. Thus, compared with classical dense **MAES**, the proposed mechanism avoids both the $O(n^3)$ dense matrix-update cost and the $O(n^2)$ storage requirement. Compared with **fMAES** and **MADFO**, it avoids the $O(n^2)$ dense affine-scaling storage and the $O(n^2)$ matrix-vector products needed to generate mutation directions. Compared with **LMAES**, it avoids storing and applying m evolution-path vectors and keeps the scaling update purely diagonal. The main trade-off is that the proposed scaling is diagonal and therefore less expressive than full or limited-memory matrix adaptation. Nevertheless, this trade-off is attractive for high-dimensional noisy DFO problems, where dense matrix adaptation may be computationally prohibitive and noisy function values may make aggressive full-matrix learning unreliable. In summary, the proposed strategy replaces the dense affine-scaling matrix A_k in **MAES**-type methods by the diagonal inverse-square-root scaling matrix

$$D_k = \mathcal{P}_k^{-1/2}.$$

Consequently, the resulting mutation distribution is

$$\mathcal{N}(\mathbf{x}_k^{\text{rec}}, \sigma_k^2 \mathcal{P}_k^{-1}),$$

which incorporates derivative-free curvature information into the mutation phase while avoiding explicit gradient evaluations, Hessian evaluations, and dense affine-scaling updates.

Table 2: Internal cost comparison of mutation-scaling mechanisms, excluding objective-function evaluations. Here, n is the problem dimension, λ is the population size, and $m \ll n$ is the number of stored evolution-path vectors in **LMMAES**.

Solver	Storage	Mutation generation	Scaling update
Classical MAES	$O(n^2)$	$O(\lambda n^2)$	$O(n^3)$
fMAES	$O(n^2)$	$O(\lambda n^2)$	$O(n^2)$
MADFO	$O(n^2)$	$O(\lambda n^2)$	$O(n^2)$
LMMAES	$O(mn)$	$O(\lambda mn)$	$O(mn)$
DMADFO	$O(n)$	$O(\lambda n)$	$O(n)$

3 A Comparison Among **DMADFO** Variants

Tables 3 and 4 summarize the performance of twelve variants of the **DMADFO** algorithm, corresponding to the parameter pairs

$$(p, z) \in \{(1, 0), (1, 1), (2, 0), (3, 0), \dots, (10, 0), (15, 0), (20, 0)\}.$$

These variants were tested on the 655 problems from the **prince** collection under noisy objective evaluations. More precisely, each test problem was extended to four noise models, namely absolute uniform, absolute Gaussian, relative uniform, and relative Gaussian noises, and six noise levels; i.e.,

$$\omega \in \{10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}.$$

Thus, the full test set contains

$$4 \times 6 \times 655 = 15720$$

noisy test instances. Table 3 reports the cumulative number of successfully solved problems, whereas Table 4 reports the number of unsolved problems classified by termination reason: reaching the maximum number of function evaluations, **nfmax**, reaching the maximum CPU time, **secmax**, or algorithmic failures, including possible bugs in the solvers. Among all variants, **DMADFO12** achieves the largest total number of solved problems, namely 10835, and the smallest number of failures due to **nfmax**, namely 4885. Moreover, no failures are caused by **secmax** or algorithmic breakdowns. Among all variants, **DMADFO12** achieves the largest total number of solved problems, namely 10835 out of 15720 test instances, corresponding to a success rate of approximately 68.90%. It also yields the smallest number of failures due to **nfmax**, namely 4885, while no failures are attributed to **secmax** or algorithmic breakdowns. Therefore, **DMADFO12** is the most robust variant among the tested versions and is selected as the representative implementation of **DMADFO** for the subsequent comparisons with competing solvers.

Table 3: Cumulative number of solved problems for all noise levels $\omega = 10^{-i}$ with $i = -2, \dots, 3$.

	Absolute/Relative uniform and Gaussian noises											
Solvers	DMADF012	DMADF08	DMADF09	DMADF011	DMADF010	DMADF05	DMADF07	DMADF06	DMADF04	DMADF03	DMADF02	DMADF01
All ϵ	Number of solved problems											
Σ	10835	10808	10807	10807	10803	10777	10775	10752	10738	10552	10519	9581
	Absolute uniform noise											
ϵ	Number of solved problems											
0.0001	2690	2686	2687	2696	2685	2680	2673	2666	2664	2631	2632	2397
	Absolute Gaussian noise											
ϵ	Number of solved problems											
0.0001	2572	2551	2550	2563	2569	2562	2559	2553	2555	2515	2541	2371
	Relative uniform noise											
ϵ	Number of solved problems											
0.0001	2805	2819	2804	2790	2803	2776	2793	2791	2780	2720	2695	2415
	Relative Gaussian noise											
ϵ	Number of solved problems											
0.0001	2768	2752	2766	2758	2746	2759	2750	2742	2739	2686	2651	2398

Table 4: Number of unsolved problems classified by termination reason—Reaching the maximum number of function evaluations (**nfmax**), reaching the maximum CPU time (**secmax**), and algorithmic failures, including possible solver bugs.

Solver	DMADF012	DMADF08	DMADF09	DMADF011	DMADF010	DMADF05	DMADF07	DMADF06	DMADF04	DMADF03	DMADF02	DMADF01
# nfmax	4885	4912	4913	4913	4917	4943	4945	4968	4982	5168	5201	6139
# secmax	0	0	0	0	0	0	0	0	0	0	0	0
# alg	0	0	0	0	0	0	0	0	0	0	0	0
# total	4885	4912	4913	4913	4917	4943	4945	4968	4982	5168	5201	6139

4 Tables for Comparison of the Most Robust DMADF0 Variant with MADF0

Tables 5 and 6 compare the performance of **MADF0** and the most robust variant of **DMADF0**, namely **DMADF012**, on the same set of 15720 noisy test instances. Overall, **MADF0** successfully solves 11112 problems, whereas **DMADF012** solves 10835 problems. Thus, **DMADF012** solves only 2.49% fewer problems than **MADF0** and retains 97.51% of its solving capability. Furthermore, the difference in the numbers of unsolved problems is entirely due to the **nfmax** termination criterion, while neither solver experiences failures caused by **secmax** limits or algorithmic breakdowns.

Table 5: Cumulative number of solved problems for **MADF0** and **DMADF012** over all noise levels $\omega = 10^{-i}$, $i = -2, \dots, 3$.

	Absolute/Relative uniform and Gaussian noises	
Solvers	MADF0	DMADF012
all ϵ	Number of solved problems	
Σ	11112	10835
	Absolute uniform noise	
ϵ	Number of solved problems	
0.0001	2820	2690
	Absolute Gaussian noise	
ϵ	Number of solved problems	
0.0001	2683	2572
	Relative uniform noise	
ϵ	Number of solved problems	
0.0001	2846	2805
	Relative Gaussian noise	
ϵ	Number of solved problems	
0.0001	2763	2768

Table 6: Number of unsolved problems for **MADF0** and **DMADF012** classified by termination reason—Reaching the maximum number of function evaluations, **nfmax**, reaching the maximum CPU time, **secmax**, and algorithmic failures, including possible solver bugs.

Solvers	MADF0	DMADF012
# nfmax	4608	4885
# secmax	0	0
# alg	0	0
# total	4608	4885

5 Tables for Comparison of the Most Robust **DMADF0** Variant with **LMMAES**

Tables 7 and 8 show that **DMADF012** clearly outperforms **LMMAES** on the 22224 noisy test instances. In total, **DMADF012** solves 14760 problems, whereas **LMMAES** solves only 7184 problems. Thus, **DMADF012** solves 7576 more problems than **LMMAES**, corresponding to about 105.5% more solved problems. Equivalently, **DMADF012** solves 66.4% of all noisy test instances, while **LMMAES** solves only 32.3%. The unsolved-problem statistics in Table 8 confirm the same conclusion: **DMADF012** terminates due to **nfmax** in 7464 cases, whereas **LMMAES** reaches **nfmax** in 15040 cases. Hence, the diagonal approximate covariance matrix used by **DMADF012** appears substantially more robust than the limited-memory covariance approximation used by **LMMAES** on this large-scale noisy test set.

Table 7: Cumulative number of solved problems for DMADF012 and LMMAES over all noise levels $\omega = 10^{-i}$, $i = -2, \dots, 3$.

	Absolute/Relative uniform and Gaussian noises	
Solvers	DMADF012	LMMAES
All ϵ	Number of solved problems	
Σ	14760	7184
	Absolute uniform noise	
ϵ	Number of solved problems	
0.0001	3678	1909
	Absolute Gaussian noise	
ϵ	Number of solved problems	
0.0001	3554	1884
	DMADF012	LMMAES
all ϵ	number of solved problems	
Σ	14760	7184
	Absolute uniform noise	
ϵ	Number of solved problems	
0.0001	3678	1909
	Absolute Gaussian noise	
ϵ	Number of solved problems	
0.0001	3554	1884
	Relative uniform noise	
ϵ	Number of solved problems	
0.0001	3784	1702
	Relative Gaussian noise	
ϵ	Number of solved problems	
0.0001	3744	1689

Table 8: Number of unsolved problems for DMADF012 and LMMAES classified by termination reason—Reaching the maximum number of function evaluations, **nfmax**, reaching the maximum CPU time, **secmax**, and algorithmic failures, including possible solver bugs.

Solvers	DMADF012	LMMAES
# nfmax	7464	15040
# secmax	0	0
# alg	0	0
# total	7464	15040

References

- [1] Anne Auger and Nikolaus Hansen. A restart CMA evolution strategy with increasing population size. In Proceedings of the 2005 IEEE Congress on Evolutionary Computation (CEC 2005), pages 1769–1776. IEEE, 2005.
- [2] Hans-Georg Beyer. Design principles for matrix adaptation evolution strategies. In Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion, GECCO 20, page 682–700. ACM, 2020.
- [3] Morteza Kimiaei and Arnold Neumaier. Effective matrix adaptation strategy for noisy derivative-free optimization. Mathematical Programming Computation, 16(3):459–501, July 2024.
- [4] Ilya Loshchilov, Tobias Glasmachers, and Hans-Georg Beyer. Large scale black-box optimization by limited-memory matrix adaptation. IEEE Transactions on Evolutionary Computation, 23(2):353–358, April 2019.